

LECTURE 31

File Systems: Directories, Protection, and Memory-Mapped Files

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Directory implementation: linear lists vs. hash tables, and why real file systems overwhelmingly choose the latter
- File protection: access lists vs. the condensed owner/group/other permission model UNIX-like systems actually use
- Reading and constructing concrete rwx permission strings and their numeric chmod-style equivalents
- Memory-mapped files: what `mmap()`-style mapping does, why it is often faster and simpler than explicit `read()/write()`, and when to reach for it

Directory Implementation

A directory is, underneath its tree-structured appearance, just another data structure that has to be searched every time a file is created, opened, renamed, or deleted. Two implementations dominate in practice.

Linear list

A simple list of file names with pointers to their data blocks — easy to implement, but every lookup, creation, or deletion scans the whole list: $O(n)$

Hash table

A hash of the file name selects (almost directly) its entry — near $O(1)$ average-case lookup, the structure real file systems actually use

Linear List vs. Hash Table, in Practice

- A linear list has to be scanned from the start, on every single operation, to find a match or confirm a name is not already taken — unnoticeable for a handful of files, but every `create()` degrades toward scanning the entire list as a directory grows to tens of thousands of entries
- A hash table fixes this directly: the file name is run through a hash function that produces (close to) the position of its entry immediately, turning lookup, insertion, or deletion into a near-constant-time operation on average
- The usual caveat applies: collisions — two different file names hashing to the same slot — have to be resolved (commonly by chaining), and a poorly sized or poorly distributed hash table can degrade back toward linear-list behavior in the worst case

This is the same trade-off a data structures course already covers in depth

A directory's lookup problem — many keys, frequent membership tests, frequent insertions and deletions — is exactly the problem hash tables were built to solve, and the same $O(1)$ -average-but- $O(n)$ -worst-case trade-off applies here as it does anywhere else a hash table is used.

File Protection: Access Lists

Once a file exists inside a directory, the file system has to decide who, exactly, is allowed to do what to it. Two broad approaches exist — starting with the most flexible.

- An access list attaches to each file an explicit list of exactly which users (or groups) may perform exactly which operations on it — one entry per user, each naming its own permitted operations
- This is maximally flexible: a file can be shared with precisely the five specific people who need it, with a different permission for each of them if required
- The flexibility has a cost: a file shared across a large, changing population of users can accumulate an unwieldy, hard-to-audit list, and every new user who needs access means another entry to add (and, eventually, remove again)

The Condensed Owner/Group/Other Scheme

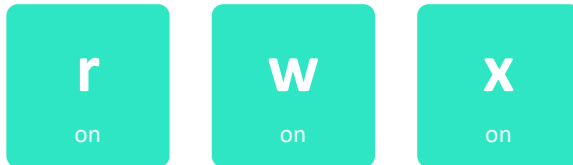
Real UNIX-like systems (Linux, macOS, and the file systems they rely on) use a far more compact scheme instead: every file carries exactly three sets of permission bits — no per-user list at all.

- Owner — the user who owns the file
- Group — a set of users sharing group membership with the file
- Other — everyone else
- Each of the three sets carries the same three bits: r (read) — permission to read the file's contents (or list a directory's entries)
- w (write) — permission to modify the file's contents (or create/delete entries within a directory)
- x (execute) — permission to run the file as a program (or to enter/traverse a directory)

Reading a Permission String: `rwxr-xr--`

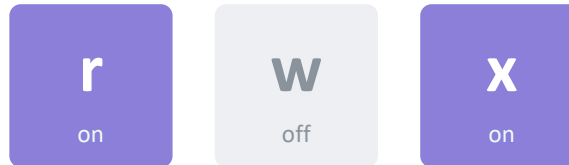
Reading `rwxr-xr--` left to right, in groups of three: the owner gets `rwx` (full access); the group gets `r-x` (read and execute, not write); other gets `r--` (read-only).

Owner



$$4+2+1 = 7$$

Group



$$4+0+1 = 5$$

Other



$$4+0+0 = 4$$

`rwxr-xr--` is equivalent to the numeric mode **754** (`chmod 754 report.sh`)

The Numeric (chmod-style) Form

- Because three bits map naturally onto three binary digits, each rwx triplet is also commonly written as a single octal digit — $r=4$, $w=2$, $x=1$, summed together for whichever permissions are actually granted
- `rwxr-xr--` becomes 754: owner $r(4)+w(2)+x(1)=7$, group $r(4)+x(1)=5$, other $r(4)=4$ — exactly the value a `chmod 754 report.sh` command sets directly, in one step

A quick way to sanity-check a numeric mode

Any single digit from 0–7 is just a 3-bit number: break it into read (4), write (2), and execute (1), and check which bits are actually set. 6 is 4+2 — read and write, no execute (common for a plain data file); 5 is 4+1 — read and execute, no write (common for a shared, non-editable script); 0 means no permission at all.

Owner/Group/Other vs. a Full Access List

- Collapsing arbitrary per-user permissions down to exactly three fixed sets is precisely what makes owner/group/other compact — and precisely what limits it
- If a file genuinely needs five specific, unrelated users to each have a different permission, three fixed buckets cannot express that directly

Owner/group/other is strictly less expressive than a full access list

Real systems that need finer-grained control layer an actual access-control list (an ACL) back on top of this base scheme — exactly the model Lecture 32's look at NTFS permissions returns to.

Memory-Mapped Files

Everything so far has assumed a program touches a file's data through explicit `read()` and `write()` calls, each one copying data between the file and a buffer the program supplies. Memory-mapped files offer a different way entirely: a system call in the `mmap()` family maps a file's contents directly into a region of the calling process's virtual address space.

Explicit `read()/write()`

Kernel copies data from the page cache into a user-supplied buffer, and back again on write — an extra copy each way

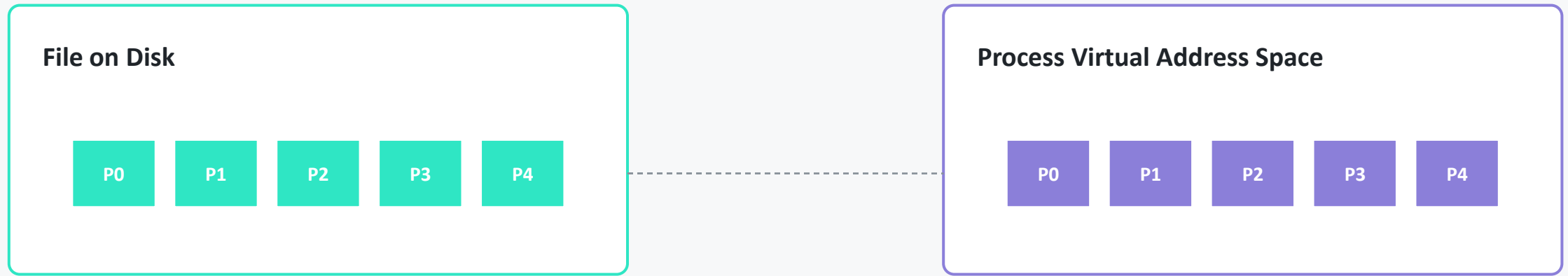


Memory-mapped file (`mmap()`)

The file's pages ARE the process's memory pages; a pointer dereference reads/writes the page cache directly — no separate buffer, no extra copy

A File's Pages, Mapped into a Process's Address Space

Mapping a file does not immediately read its contents — it reserves the address range and records what each page corresponds to on disk. The first access to a page triggers a page fault, serviced through the page cache, exactly like a demand-paged program page.



Pages map 1:1, in order, onto the same offsets within the file

No read()/write() call, no separate buffer — a pointer dereference inside the mapped region IS the file access, serviced lazily by the page cache.

Why Memory Mapping Is Often Faster and Simpler

- No extra buffer-copying — an explicit `read()` typically copies data from the kernel's page cache into a buffer the program supplies; a memory-mapped file skips that copy entirely, since the mapped pages ARE the page-cache pages
- Reuse of machinery the OS already has — lazy loading, caching of recently used pages, and eviction under memory pressure are all already implemented by the demand-paging system, for free
- A natural fit for a program needing fast, read-only access to a large data file (a multi-gigabyte lookup table), or for two processes mapping the same file to share memory directly

Memory mapping does not bypass protection

A memory-mapped file still respects the owner/group/other permissions from earlier in this lecture — a process can only map a file for writing if it already had write permission on that file to begin with. Memory mapping changes how access happens, not whether it is allowed.

Key Takeaways

- Directory implementation is usually a hash table in practice (near- $O(1)$ average lookup), not the simpler but $O(n)$ linear list
- File protection ranges from fully flexible but unwieldy access lists to the compact owner/group/other, rwx-per-set scheme real UNIX-like systems actually use — a string like `rw-r--r--` decodes directly into a numeric mode like `754`
- Memory-mapped files map a file's contents directly into a process's virtual address space, so ordinary pointer reads and writes transparently become file I/O, serviced lazily through the same demand-paging machinery that handles regular program pages
- Memory mapping is often faster than explicit `read()/write()` because it avoids an extra buffer copy and reuses the OS's existing page-cache and eviction logic — and it doubles as a natural mechanism for processes to share memory by mapping the same file